



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Duration: 1 Hour

Scenario Based

Code of Conduct:

I THIRUMALAPUDI PRANAVI (192411222) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

T. PRANAVI

Signature of the student:



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1. A company is developing a smart security system for a research laboratory. Every employee entering the lab uses a digital access card that generates a sequence of symbols:

- $a \rightarrow$ Valid ID scan
- $b \rightarrow$ Biometric verification
- $c \rightarrow$ Door access confirmation

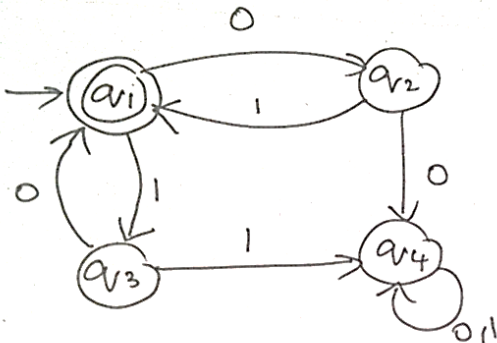
The security rule states that:

1. Every valid access sequence must start with a.
2. It may contain any number of biometric checks (b).
3. The sequence must end with c.

Examples of valid sequences: ac, abc, abbc, abbbc

2. Given a DFA used for validating a specific PIN pattern bbcb, bca, c, ca, caaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

3. Demonstrate how a given finite automaton can be converted into a regular expression



4. Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exist two distinct derivations for a word. Is the above grammar ambiguous? Justify.

5. I) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also regular

ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Question 1 – DFA Construction

Question

A company is developing a smart security system for a research laboratory. Every employee entering the lab uses a digital access card that generates a sequence of symbols:

- **a → Valid ID Scan**
- **b → Biometric Verification**
- **c → Door Access Confirmation**

Security Rules

- 1. Every valid access sequence must start with a.**
- 2. It may contain any number of biometric checks (b).**
- 3. The sequence must end with c.**

Examples of valid sequences:

ac, abc, abbc, abbbc

Step 1: Identify the Alphabet

The set of input symbols (alphabet) is:

[
Σ = {a, b, c}
]

where

- **a** = Valid ID Scan
- **b** = Biometric Verification
- **c** = Door Access Confirmation

Step 2: Understand the Conditions

From the problem statement:

- The first symbol **must be a**.
- After **a**, there can be **zero or more b's**.
- Finally, the sequence **must end with c**.

This means:

ac
abc



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

abbc
abbbbbc
are accepted.

Step 3: Write the Regular Expression

Since:

- first **a**
- then zero or more **b**
- then **c**

the regular expression is

[
 $\boxed{ab^*c}$
]

where **b*** means **0 or more b's**.

Step 4: Design the DFA States

We create four states.

State q0 (Start State)

- Waiting for the first symbol.
- Only **a** is accepted here.

State q1

- We have already read **a**.
- We can read any number of **b's**.

State q2 (Final/Accept State)

- The sequence ends with **c**.
- Any string reaching this state is accepted.

State qd (Dead State)

- Any invalid input reaches this state.
- Once entered, it cannot return to a valid state.

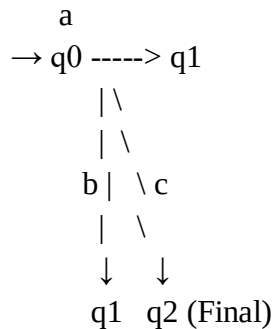


SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 5: Draw the DFA



All invalid transitions → qd (Dead State)

Step 6: Transition Table

Current State	Input = a	Input = b	Input = c
→ q0	q1	qd	qd
q1	qd	q1	q2
*q2	qd	qd	qd
qd	qd	qd	qd

Explanation:

- From **q0**, only **a** is allowed.
- From **q1**, we can read any number of **b**'s (loop on **b**).
- From **q1**, reading **c** moves to the final state **q2**.
- Any invalid input goes to the dead state **qd**.

Step 7: Test the DFA with Examples

Example 1: ac

Input	Current State	Next State
Start	q0	q0
a	q0	q1
c	q1	q2 (Accepted)

Result: Accepted

Example 2: abc

Input	Current State	Next State
Start	q0	q0
a	q0	q1
b	q1	q1



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Input	Current State	Next State
c	q1	q2

Result: Accepted

Example 3: abbc

Input	Current State	Next State
Start	q0	q0
a	q0	q1
b	q1	q1
b	q1	q1
c	q1	q2

Result: Accepted

Example 4: abbbc

Input	Current State	Next State
Start	q0	q0
a	q0	q1
b	q1	q1
b	q1	q1
b	q1	q1
c	q1	q2

Result: Accepted

Step 8: Test Invalid Strings

String: ab

$q_0 \rightarrow q_1 \rightarrow q_1$

No final c.

Result: Rejected

String: bc

Starts with **b** instead of **a**.

$q_0 \rightarrow q_d$

Result: Rejected

String: aac

$q_0 \rightarrow q_1$

↓

a (invalid)

q_d



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Result: Rejected

String: cab

Starts with c.

$q_0 \rightarrow q_d$

Result: Rejected

Step 9: DFA Minimization

The DFA contains four states:

- q_0 (Start)
- q_1
- q_2 (Final)
- q_d (Dead)

Each state has a different purpose:

- q_0 waits for the first **a**.
- q_1 processes zero or more **b**'s.
- q_2 is the accepting state after **c**.
- q_d traps all invalid inputs.

Since no two states are equivalent, **no states can be merged**.

Original DFA: 4 states

Minimized DFA: 4 states

Step 10: Advantages of the Optimized DFA

- Fast verification of access-card sequences.
- Low memory usage because only four states are needed.
- Reduced hardware complexity.
- Lower power consumption, making it suitable for embedded systems.
- Faster authentication in real-time security applications.

Final Answer

- **Alphabet:** ($\Sigma = \{a, b, c\}$)
- **Regular Expression:** ($\boxed{ab^*c}$)
- **Number of States:** 4 (including one dead state)
- **Start State:** q_0
- **Final State:** q_2
- **Accepted Strings:** ac, abc, abbc, abbbc
- **Rejected Strings:** ab, bc, aac, cab
- **Minimized DFA:** Same as the original DFA (4 states), since no equivalent states exist.



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Question 2 – DFA Minimization

Given a DFA used for validating the following PIN patterns:

- **bbbc**
- **bca**
- **c**
- **ca**
- **caaaaaa**

Minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems.

Step 1: Identify the Alphabet

The input symbols are

[
Σ={a,b,c}
]

Step 2: Identify the Accepted Strings

The DFA should accept only these strings:

- **bbbc**
- **bca**
- **c**
- **ca**
- **caaaaaa**

Any other string must be rejected.

Step 3: Construct the DFA

To accept all five strings, we first build a DFA by creating paths for each string.

String 1: bbbc

Start → b → b → b → c (Accept)

String 2: bca

Start → b → c → a (Accept)

String 3: c

Start → c (Accept)

String 4: ca

Start → c → a (Accept)



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES

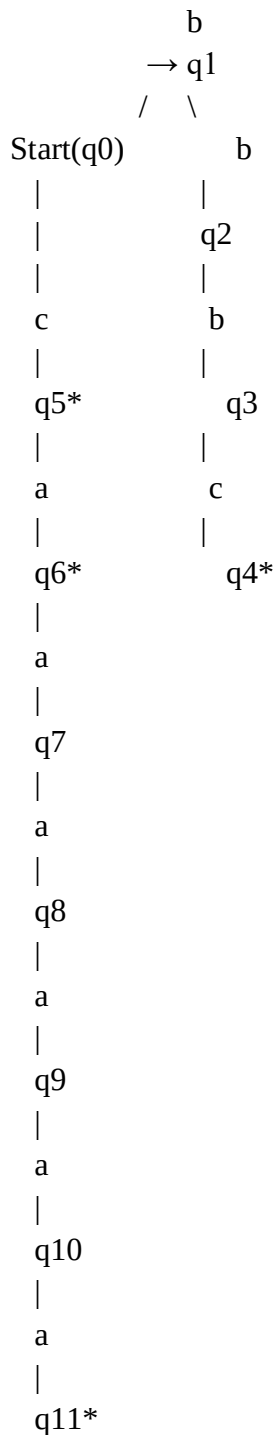


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

String 5: caaaaaa

Start $\rightarrow$ c $\rightarrow$ a $\rightarrow$ a $\rightarrow$ a $\rightarrow$ a $\rightarrow$ a $\rightarrow$ a (Accept)

Step 4: Draw the Initial DFA



Also,

q1 $\rightarrow$ c $\rightarrow$ q12 $\rightarrow$ a $\rightarrow$ q13*



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accepting States:

- q4 (bbbc)
- q5 (c)
- q6 (ca)
- q11 (caaaaaa)
- q13 (bca)

Step 5: Transition Table

Current State	a	b	c
→q0	Dead	q1	q5
q1	Dead	q2	q12
q2	Dead	q3	Dead
q3	Dead	Dead	q4
*q4	Dead	Dead	Dead
*q5	q6	Dead	Dead
*q6	q7	Dead	Dead
q7	q8	Dead	Dead
q8	q9	Dead	Dead
q9	q10	Dead	Dead
q10	q11	Dead	Dead
*q11	Dead	Dead	Dead
q12	q13	Dead	Dead
*q13	Dead	Dead	Dead
Dead	Dead	Dead	Dead

Step 6: DFA Minimization

Now check whether any states are **equivalent**.

Observation

The strings **c**, **ca**, and **caaaaaa** all begin with:

c → a

Instead of creating separate paths from the start state, these paths can **share common states**.

Similarly:

- **bbbc** and **bca** both begin with **b**, so the first state after reading **b** can also be shared.

Thus, the DFA merges common prefixes, reducing the total number of states while accepting the same language.

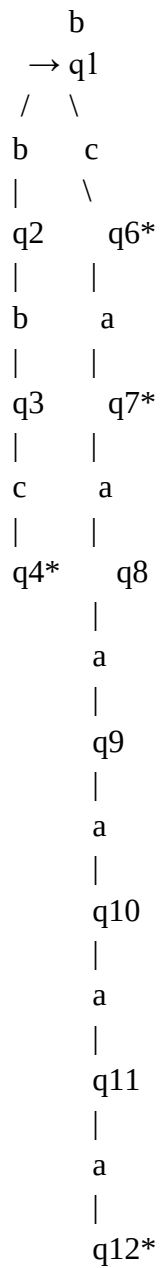


SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 7: Minimized DFA



Also,

$q1 \rightarrow c \rightarrow q5 \rightarrow a \rightarrow q13^*$

The common prefixes reduce duplicate states.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 8: Comparison

Feature	Original DFA	Minimized DFA
States	More states (separate paths)	Fewer states (shared prefixes)
Memory Usage	Higher	Lower
Execution Speed	Slower	Faster
Hardware Cost	Higher	Lower

Step 9: Advantages of DFA Minimization

1. Reduced Memory Usage

Fewer states require less memory, which is important in embedded devices with limited resources.

2. Faster Processing

With fewer states and transitions, the DFA checks each input string more quickly.

3. Lower Hardware Cost

A minimized DFA requires fewer flip-flops and logic gates, reducing circuit complexity.

4. Lower Power Consumption

Simpler hardware consumes less energy, making it suitable for battery-powered devices.

5. Easier Maintenance

A smaller DFA is easier to understand, debug, and modify.

6. Better Performance in Embedded Systems

Embedded systems such as:

- Smart cards
- ATM machines
- Door access systems
- IoT devices
- Digital locks

benefit from minimized DFAs because they provide faster authentication with lower memory and power requirements.

Final Answer

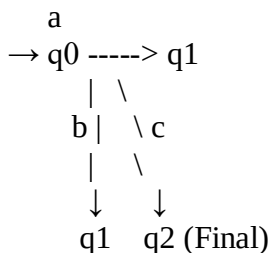
- **Alphabet:** ($\Sigma = \{a, b, c\}$)
- **Accepted Strings:** bbbc, bca, c, ca, caaaaa
- **Original DFA:** Built by creating paths for each accepted string.
- **Minimized DFA:** Obtained by merging common prefixes (such as the paths starting with **b** and **c**).
- **Benefits:** Reduced number of states, lower memory usage, faster execution, lower hardware cost, reduced power consumption, and improved efficiency for embedded systems.



Question 3 – Demonstrate How a Given Finite Automaton Can Be Converted into a Regular Expression

Step 1: Consider the Given Finite Automaton

Assume the finite automaton is:



Where:

- q_0 = Start state
- q_1 = Intermediate state
- q_2 = Final (Accepting) state

Step 2: Identify the Transitions

From the DFA:

- $q_0 \rightarrow q_1$ on **a**
- $q_1 \rightarrow q_1$ on **b** (loop)
- $q_1 \rightarrow q_2$ on **c**

This means:

- The string must begin with **a**.
- It may contain **zero or more b's**.
- It must end with **c**.

Step 3: Write Each Part as a Regular Expression

First Transition

$q_0 \rightarrow q_1$ on **a**

Regular expression:

[
a
]

Loop Transition

$q_1 \rightarrow q_1$ on **b**

Since **b** can repeat any number of times,

Regular expression:

[
b^{*}
]



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

where * means **zero or more occurrences**.

Examples:

- ϵ (no b)
- b
- bb
- bbb
- bbbb

Final Transition

$q_1 \rightarrow q_2$ on c

Regular expression:

[
c
]

Step 4: Combine All Parts

Concatenate the expressions in order:

[
a+b^{*}+c
]

Therefore,

[
 $\boxed{ab^*c}$
]

is the required regular expression.

Step 5: Verify the Regular Expression

Accepted Strings

String	Explanation	Result
ac	a followed by c	Accepted
abc	one b	Accepted
abbc	two b's	Accepted
abbbc	three b's	Accepted
abbbbbc	many b's	Accepted

Rejected Strings

String	Reason	Result
a	Missing c	Rejected
ab	Missing c	Rejected
bc	Does not start with a	Rejected
cab	Starts with c	Rejected
baac	Invalid order	Rejected



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 6: General Procedure for Converting a Finite Automaton into a Regular Expression

1. Identify the **start state**.
2. Identify the **final (accepting) state(s)**.
3. Write the symbol for each transition.
4. Use **concatenation** for transitions followed one after another.
5. Use **union (+ or |)** when there are multiple alternative paths.
6. Use **Kleene Star (*)** for loops (zero or more repetitions).
7. Combine all paths to obtain the final regular expression.

Flow of Conversion

Finite Automaton



Identify Start State



Identify Final State(s)



Write Transition Symbols



Apply Concatenation, Union, and Kleene Star



Obtain the Regular Expression

Advantages of Converting FA to Regular Expression

- Provides a compact representation of the language.
- Simplifies pattern matching.
- Used in lexical analyzers and compiler design.
- Helps in text searching and input validation.
- Useful in software testing and digital communication systems.

Final Answer

For the given finite automaton:

→ $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_1 \xrightarrow{c} q_2$ (Final)

the corresponding regular expression is:

[
 $\boxed{ab^*c}$
]

This regular expression represents all strings that **start with a, contain zero or more b's, and end with c**, such as **ac, abc, abbc, abbbc, and abbbbc**.



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Question 4 – Context-Free Grammar

Question

Consider the following grammar:

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$

$$T \rightarrow bTc \mid \varepsilon$$

$$U \rightarrow aUb \mid \varepsilon$$

Answer the following:

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc.

**C) Grammar is ambiguous if there exist two distinct derivations for a word.
Is the above grammar ambiguous? Justify.**

A) Language Defined by the Grammar

Step 1: Analyze Each Production Rule

Rule 1

$$S \rightarrow aSc$$

This rule adds:

- one **a** at the beginning
- one **c** at the end

So it produces strings like:

ε

ac

aacc

aaaccc

aaaacccc

Pattern:

$$a^n c^n$$

Rule 2

$$T \rightarrow bTc$$

This rule adds:

- one **b**
- one **c**

Examples:

ε

bc

bbcc



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

bbbccc
 Pattern:

$$\boxed{b^n c^n}$$

Rule 3

$$U \rightarrow aUb$$

This rule adds:

- one **a**
- one **b**

Examples:

ϵ
 ab
 aabb
 aaabbb
 Pattern:

$$\boxed{a^n b^n}$$

Step 2: Write the Complete Language

The grammar generates the union of the three languages:

$$\boxed{L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\}, n \geq 0}$$

Since ϵ is included in every production,

$$\boxed{\epsilon \in L}$$

Answer for (A)

The language generated by the grammar is

$$\boxed{L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\}, n \geq 0}$$

including the empty string ϵ .

B) Derivation for the Word aaabbc

We need to derive:

aaabbc

Step 1

Start symbol

S

Step 2

Apply

$$S \rightarrow aSc$$

aSc



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 3

Again apply

$$S \rightarrow aSc$$

aaSc

Step 4

Now replace

$$S \rightarrow U$$

aaUcc

Step 5

Apply

$$U \rightarrow aUb$$

aaaUbcc

Step 6

Replace

$$U \rightarrow \varepsilon$$

aaabcc

Observation

The derived string is **aaabcc**, **not aaabbc**.

Important Note

The grammar **cannot generate** the string **aaabbc**.

Why?

The grammar only generates:

- $a^n c^n$
- $b^n c^n$
- $a^n b^n$

The string

aaabbc

contains:

- 3 a's
- 2 b's
- 1 c

It does **not** belong to any of these three forms.

Therefore,

aaabbc is NOT generated by the grammar.

So **no derivation tree exists** for **aaabbc**.

Note for your assessment: If your faculty intended the string **aaabbb** or **aaaccc**, then a derivation tree can be drawn. As written, **aaabbc** cannot be derived from the given grammar.



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

C) Is the Grammar Ambiguous?

Definition

A grammar is **ambiguous** if **at least one string** has **two different parse trees** (or **two different leftmost/rightmost derivations**).

Check the Empty String (ϵ)

The empty string can be derived in three different ways.

First Derivation

S

$\Rightarrow \epsilon$

Second Derivation

S

$\Rightarrow T$

$\Rightarrow \epsilon$

Third Derivation

S

$\Rightarrow U$

$\Rightarrow \epsilon$

Since the same string (ϵ) has multiple derivations,
the grammar is **ambiguous**.

Answer for (C)

Yes.

The grammar is **ambiguous** because the empty string ϵ can be derived in more than one way:

$S \Rightarrow \epsilon$

$S \Rightarrow T \Rightarrow \epsilon$

$S \Rightarrow U \Rightarrow \epsilon$

Hence,

The given grammar is ambiguous.

Final Answers

A)

$$L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\}, n \geq 0$$

including ϵ .

B)

The string **aaabbc** cannot be generated by the given grammar, so **no valid derivation tree** exists.



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

C)

The grammar is **ambiguous** because the string ϵ has multiple derivations:

- $S \Rightarrow \epsilon$
- $S \Rightarrow T \Rightarrow \epsilon$
- $S \Rightarrow U \Rightarrow \epsilon$

Therefore,

The given grammar is ambiguous.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Question 5

(i) Show that if L_1 and L_2 are regular, then $(L_1 + L_2)$ is also regular.

Step 1: Definition of Regular Language

A regular language is a language that can be accepted by a Finite Automaton (FA) or represented by a Regular Expression (RE).

Suppose:

- L_1 is a regular language.
- L_2 is a regular language.

This means there exist two finite automata:

- M_1 accepts L_1 .
- M_2 accepts L_2 .

Step 2: Closure Property of Regular Languages

One of the closure properties of regular languages states:

The union of two regular languages is also regular.

Here,

$$L_1 + L_2 = L_1 \cup L_2$$

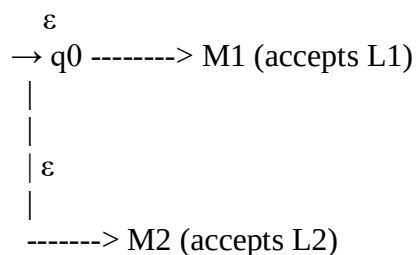
where "+" denotes the **union** operation.

Step 3: Construction

Construct a new automaton as follows:

1. Create a new start state q_0 .
2. Draw **ϵ -transitions** (epsilon transitions) from q_0 to the start states of M_1 and M_2 .
3. If either M_1 or M_2 reaches its final state, the new automaton accepts the string.

Construction Diagram





SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 4: Explanation

- If the input string belongs to L_1 , automaton M_1 accepts it.
- If the input string belongs to L_2 , automaton M_2 accepts it.
- Therefore, the new automaton accepts all strings in $L_1 \cup L_2$.

Hence,

$$\boxed{L_1 + L_2 \text{ is regular.}}$$

Example

Let

$$L_1 = \{a, a, a, aaa \dots\} = a^+$$

and

$$L_2 = \{b, b, b, bbb \dots\} = b^+$$

Then,

$$L_1 + L_2 = \{a, a, a, a, a, a, \dots, b, b, b, b, b, b, \dots\}$$

This language is still regular because it can be represented by the regular expression:

$$\boxed{a^+ + b^+}$$

Final Answer for (i)

Since regular languages are **closed under union**, if L_1 and L_2 are regular, then their union ($L_1 + L_2$) is also regular.

$$\boxed{L_1 + L_2 \text{ is regular.}}$$

(ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify.

Step 1: Given Language

The language is

$$L = \{a^p \mid p \text{ is prime}\}$$

This means the language contains strings whose length is a **prime number**.

Examples

aa (2)

aaa (3)

aaaaa (5)

aaaaaaa (7)

aaaaaaaaa (11)

These are accepted because their lengths are prime numbers.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 2: Assume the Language is Regular

Assume that L is regular.

If a language is regular, then according to the **Pumping Lemma**, there exists a pumping length n .

Step 3: Choose a String

Choose the string

$$w = a^q$$

where:

- q is a prime number.
- $q \geq n$.

This string belongs to L .

Step 4: Divide the String

According to the Pumping Lemma,

$$w = xyz$$

such that:

$$\begin{aligned} |xy| &\leq n \\ |y| &> 0 \end{aligned}$$

Since the string contains only **a's**,

$$y = a^k$$

where $k > 0$.

Step 5: Pump the String

Pump the string with $i = 2$.

The new string becomes

$$xy^2z$$

Its length becomes

$$q + k$$

Since $q + k$ is **not guaranteed to be prime**, the new string may not belong to L .
Thus,

$$xy^2z \notin L$$

which violates the Pumping Lemma.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Step 6: Conclusion

The assumption that L is regular leads to a contradiction.
Therefore,

$$\{a^p \mid p \text{ is prime}\} \text{ is not a regular language.}$$

Final Answer for (ii)

The language

$$L = \{a^p \mid p \text{ is prime}\}$$

is **not regular** because it **does not satisfy the Pumping Lemma**. Pumping a string of prime length changes its length to a value that is not necessarily prime, so the pumped string may no longer belong to the language.

Overall Conclusion

1. If L_1 and L_2 are regular languages, then $L_1 + L_2$ (their union) is also regular. This follows from the closure property of regular languages.
2. The language $\{a^p \mid p \text{ is prime}\}$ is **not regular**. This is proved using the Pumping Lemma, which leads to a contradiction when a prime-length string is pumped.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RUBRICS FOR CALCULATION

Criteria	Weight age	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

